

Trilha Java + Spring Boot + Engenharia de Software - Plano de Estudos Técnico para Nível Avançado/Sênior

Plano de estudos técnico para evolução avançada e sênior em backend Java, Spring Boot, arquitetura, sistemas distribuídos, qualidade e liderança técnica.

Documento formatado a partir do conteúdo anexado pelo usuário.
Data de geração: 06/06/2026

Sumário

Trilha Java + Spring Boot + Engenharia de Software - Plano de Estudos Técnico para Nível Avançado/Sênior 7

1. Análise inicial do conteúdo 7

Tema central	7
Objetivo principal da trilha	7
Formar um desenvolvedor Java/Spring capaz de:	7
Competências técnicas que devem ser desenvolvidas	7
Classificação por nível	8
Básico	8
Intermediário	8
Avançado	8
Sênior	9

2. Explicação técnica por tópico 9

1. Java moderno e fundamentos avançados 9

O que é	9
Para que serve	9
Records	9
Sealed classes	10
Pattern matching	10
Switch expressions	10
Text blocks	11
Optional bem aplicado	11
Streams avançado e Collectors customizados	11
Generics, wildcards e bounded types	12
Reflection e annotations customizadas	12
Class loading	12
Concorrência, virtual threads e structured concurrency	13
Virtual threads	13

2. Spring Boot profissional 15

O que é	15
Para que serve	15
Bean lifecycle	16
Profiles permitem separar configurações por ambiente:	16
Actuator, health checks e metrics	16
Validation	17
Exception handling e RFC 7807	17
Schedulers e async processing	17

Cache abstraction	18
Feign, WebClient e integrações	18
Resilience4j implementa padrões de resiliência:	18
Onde é usado em projetos reais	18
3. API Design avançado	19
O que é	19
Para que serve	19
Paginação offset vs cursor	20
Versionamento	21
HATEOAS conceitual	21
HTTP status codes	21
Backward compatibility e deprecation strategy	22
4. Persistência e banco com Java	23
O que é	23
Para que serve	23
Persistence context e dirty checking	23
Lazy/eager loading e N+1	23
Transações, isolation levels e propagation	24
Isolation levels controlam visibilidade entre transações:	24
Flyway	24
Query tuning e índices	25
Onde é usado em projetos reais	25
5. Arquitetura de software	26
O que é	26
Para que serve	26
DDD	27
Aggregates	27
Value Objects	28
Domain Events	28
Modular Monolith	28
Microservices	28
Event-driven architecture	28
ADRs e C4 Model	29
6. System Design avançado	30
O que é	30
Para que serve	30
Load balancing	30
Padrões:	31
Sharding e consistent hashing	31

Consensus, Raft, Paxos e leader election	32
CRDTs	32
Multi-region, disaster recovery, RPO e RTO	32
Fan-out on write vs fan-out on read	32
WebSocket, SSE e push notifications	32
7. gRPC e Protocol Buffers	33
O que é	33
Para que serve	33
Interceptors, auth e tracing	34
Interceptors permitem cross-cutting concerns:	34
8. Kafka avançado, Kafka Streams e Flink	35
O que é	35
Para que serve	35
Partitions e offset management	35
Idempotent producer e transactions	36
Schema Registry	36
Kafka Streams	36
9. Testes e qualidade	37
O que é	37
Para que serve	37
Mutation testing	39
ArchUnit	39
Erros comuns	39
10. Soft skills de engenharia sênior	40
O que é	40
Principais práticas	40
Code review efetivo	40
Um bom code review avalia:	40
Mentoria técnica	40
Mentoria envolve:	40
Capacidade de coordenar resposta a incidente:	41
3. Plano de estudos em 6 módulos	42
Módulo 1 - Fundamentos avançados de Java e JVM	42
Objetivo	42
Conteúdos	42
Java Runtime Lab	43
Módulo 2 - Spring Boot profissional e APIs REST	43
Objetivo	43
Conteúdos	43

Order Management API	44
Módulo 3 - Persistência, transações e qualidade	45
Objetivo	45
Conteúdos	45
Billing Persistence Lab	46
Módulo 4 - Arquitetura, DDD e modularização	46
Objetivo	46
Conteúdos	46
Modular Commerce Platform	47
Módulo 5 - Produção, performance, segurança, mensageria e escala	48
Objetivo	48
Conteúdos	48
Async Order Platform	49
Módulo 6 - Domínio sênior e liderança técnica	50
Objetivo	50
Conteúdos	50
Engineering Leadership Pack	51
4. Critérios de domínio técnico por grande área	51
Java e JVM	51
Spring Boot e APIs	52
Persistência	52
Arquitetura	52
System Design	52
Kafka, gRPC e comunicação	52
Testes e qualidade	52
5. Aplicação prática em projetos reais	53
APIs	53
6. Cronograma sugerido de execução	56
Fase 1 - Semanas 1 a 6: Java moderno e JVM	56
Prioridade	56
Estudar	56
Fase 2 - Semanas 7 a 12: Spring Boot profissional e API Design	56
Prioridade	56
Estudar	56
Fase 3 - Semanas 13 a 18: Persistência, banco e testes	57
Prioridade	57
Estudar	57
Fase 4 - Semanas 19 a 25: Arquitetura e DDD	58
Prioridade	58

Estudar	58
Fase 5 - Semanas 26 a 33: Kafka, gRPC, observabilidade e resiliência	58
Prioridade	58
Estudar	59
Fase 6 - Semanas 34 a 40: System Design e liderança técnica	59
Prioridade	59
Estudar	59
7. Projetos recomendados para portfólio	60
1. Order Management API	60
2. Billing Persistence Lab	60
3. Modular Commerce Platform	61
4. Async Order Platform	61
6. URL Shortener em escala	61
7. Notification System	62
8. Checklist de domínio técnico	62
Java	62
9. Indicadores de prontidão para nível avançado/sênior	64

Trilha Java + Spring Boot + Engenharia de Software - Plano de Estudos Técnico para Nível Avançado/Sênior

1. Análise inicial do conteúdo

Tema central

O conteúdo descreve uma trilha completa de evolução para backend Java com Spring Boot, expandida para competências de engenharia de software, arquitetura, sistemas distribuídos, mensageria, performance, qualidade, observabilidade e liderança técnica.

Não é apenas uma trilha de programação Java. É uma trilha para formar um profissional capaz de atuar em sistemas reais de produção, tomando decisões técnicas com impacto em escalabilidade, manutenção, custo, segurança e confiabilidade.

Objetivo principal da trilha

Formar um desenvolvedor Java/Spring capaz de:

- Projetar APIs, serviços e módulos backend.
- Implementar código limpo, testável e resiliente.
- Trabalhar com bancos relacionais, transações, JPA/Hibernate e tuning.
- Criar sistemas assíncronos com Kafka e arquitetura orientada a eventos.
- Aplicar arquitetura hexagonal, DDD, Clean Architecture e modularização.
- Entender system design, escalabilidade, cache, consistência e disponibilidade.
- Observar aplicações em produção com logs, métricas, tracing e profiling.
- Definir estratégia de testes, qualidade e revisão técnica.
- Liderar tecnicamente decisões, RFCs, ADRs, code reviews e incidentes.

Competências técnicas que devem ser desenvolvidas

A trilha desenvolve 10 grandes competências:

- Domínio da linguagem Java moderna
- Java 17/21, JVM, concorrência, memória, profiling e código idiomático.
- Domínio profissional do Spring Boot
- APIs, DI, lifecycle, validação, segurança, observabilidade, resiliência e integrações.
- API Design avançado
- Contratos evolutivos, versionamento, idempotência, erros padronizados e compatibilidade.
- Persistência e banco de dados
- JPA/Hibernate avançado, transações, locks, índices, pool, tuning, outbox e CDC.
- Arquitetura de software
- Clean Architecture, Hexagonal, DDD, modular monolith, microservices e eventos.
- System Design
- Escala, cache, sharding, replicação, consenso, multi-região, RPO/RTO e backpressure.
- Comunicação interserviços com gRPC
- Protobuf, streaming, deadlines, compatibilidade, tracing e comparação com REST.
- Mensageria e processamento de stream
- Kafka, Kafka Streams, Flink, DLQ, retries, Schema Registry, lag e exactly-once.

- Testes e qualidade
- Unitários, integração, contrato, carga, mutação, arquitetura e estratégia por risco.
- Soft skills de engenharia sênior
- Review, mentoria, comunicação, ADR, RFC, postmortem, incidentes e influência técnica.

Classificação por nível

Básico

O estudante precisa entender:

- Sintaxe Java.
- Orientação a objetos.
- Collections.
- Exceptions.
- HTTP básico.
- Spring Boot básico.
- CRUD com banco.
- Testes unitários simples.
- SQL básico.
- Git, Docker e REST básico.

Intermediário

O estudante precisa aplicar:

- Java moderno em código real.
- Spring Data, validação, tratamento de erro.
- APIs REST bem estruturadas.
- JPA com relacionamentos.
- Testes unitários e integração.
- Docker Compose.
- Segurança básica com JWT/OAuth2.
- Kafka básico.
- Camadas de aplicação.
- Logs e métricas iniciais.

Avançado

O estudante precisa dominar:

- Concorrência, JVM, GC e profiling.
- Spring Boot com resiliência, observabilidade e segurança.
- JPA/Hibernate avançado.
- Transações e isolamento.
- API Design evolutivo.
- Arquitetura hexagonal e DDD prático.
- Kafka com DLQ, retries e idempotência.
- Testcontainers, WireMock, Pact, k6/Gatling.
- Performance, cache e tuning.

Sênior

O estudante precisa decidir, projetar, revisar e justificar:

- Quando usar monólito modular ou microsserviços.
- Quando usar REST, gRPC, GraphQL ou eventos.
- Como desenhar sistemas escaláveis e resilientes.
- Como lidar com consistência eventual.
- Como evitar efeito cascata entre serviços.
- Como evoluir contratos sem quebrar clientes.
- Como diagnosticar incidentes de produção.
- Como documentar decisões arquiteturais.
- Como orientar times tecnicamente.

2. Explicação técnica por tópico

1. Java moderno e fundamentos avançados

O que é

É o domínio aprofundado da linguagem Java, especialmente nas versões modernas como Java 17 e Java 21, incluindo recursos de sintaxe, modelagem, concorrência, JVM, memória e diagnóstico.

Esse tópico não é apenas "saber programar em Java". É entender como a linguagem, a JVM e o runtime se comportam em produção.

Para que serve

Serve para escrever código:

- Mais expressivo.
- Mais seguro.
- Mais performático.
- Mais testável.
- Mais fácil de manter.
- Mais preparado para concorrência.
- Mais previsível em produção.
- Principais blocos de estudo
- Java 17/21 em profundidade

Java 17 é uma versão LTS amplamente usada em produção. Java 21 também é LTS e trouxe recursos importantes, especialmente virtual threads, pattern matching mais maduro e melhorias de linguagem.

O objetivo aqui é dominar Java como plataforma moderna, não como linguagem antiga baseada apenas em classes, getters, setters e herança.

Records

record é uma forma concisa de modelar dados imutáveis.

Usado para:

- DTOs.
- Value Objects simples.
- Responses.

- Commands.
- Query results.
- Objetos de transporte interno.

Evite usar records como entidades JPA complexas, porque entidades geralmente precisam de lifecycle, proxies e mutabilidade controlada pelo ORM.

Erro comum: usar record para tudo, inclusive onde há comportamento de domínio, identidade mutável ou necessidade de lazy loading.

Sealed classes

Permitem restringir quais classes podem herdar ou implementar determinado tipo.

Usado para:

- Modelar hierarquias fechadas.
- Representar estados de domínio.
- Representar tipos de erro.
- Reduzir heranças imprevisíveis.

Exemplo real: representar diferentes tipos de resultado de uma operação:

- Success
- BusinessError
- TechnicalError
- ValidationError

Valor sênior: sealed classes ajudam a tornar o domínio mais explícito e seguro.

Pattern matching

Permite verificar tipo e extrair valor de forma mais limpa.

Ajuda em:

- Tratamento de hierarquias.
- Decisões baseadas em tipo.
- Código mais legível.
- Redução de casts manuais.

Erro comum: usar pattern matching para substituir polimorfismo onde o comportamento deveria estar nas próprias classes.

Switch expressions

Permitem escrever switch como expressão retornando valor.

Bom para:

- Mapeamento de enums.
- Estratégias simples.
- Conversões controladas.
- Tratamento de estados.

Erro comum: criar switch gigante com regra de negócio complexa. Nesse caso, pode ser melhor usar Strategy, Polymorphism ou Domain Services.

Text blocks

Facilitam strings multilinha.

Usado em:

- Queries SQL.
- JSON de teste.
- XML.
- Payloads de integração.
- Documentação inline em testes.

Erro comum: colocar SQL complexo diretamente espalhado no código sem organização ou sem testes.

Optional bem aplicado

Optional representa ausência de valor em retornos.

Use para:

- Retornos de busca.
- Métodos que podem não encontrar um resultado.
- Evitar null em fluxos controlados.

Evite em:

- Campos de entidade.
- DTOs serializados.
- Parâmetros de método.
- Getters JPA.

Critério sênior: saber quando Optional melhora clareza e quando vira ruído.

Streams avançado e Collectors customizados

Streams ajudam no processamento declarativo de coleções.

Usado para:

- Filtrar.
- Mapear.
- Agrupar.
- Reduzir.
- Transformar dados.
- Criar agregações.

Collectors customizados são úteis quando a agregação não se encaixa bem em toList, groupingBy, mapping, reducing.

Erro comum: usar stream para tudo, criando pipelines ilegíveis. Código sênior prioriza clareza, não "esperteza".

Generics, wildcards e bounded types

Generics permitem criar código reutilizável com segurança de tipo.

Importante para:

- APIs reutilizáveis.
- Bibliotecas internas.
- Repositórios genéricos.
- Mappers.
- Builders.
- Estratégias de processamento.

Conceitos essenciais:

- T
- ?
- ? extends T
- ? super T
- bounded types
- type erasure

Critério sênior: entender quando generics aumentam flexibilidade e quando criam complexidade desnecessária.

Reflection e annotations customizadas

Reflection permite inspecionar classes, métodos e campos em runtime. Annotations permitem adicionar metadados ao código.

Usado em:

- Frameworks.
- Validação.
- Mappers.
- Serialização.
- Injeção de dependência.
- Testes.
- Auditoria.
- Cross-cutting concerns.

Spring usa fortemente reflection e annotations.

Erro comum: criar muita "mágica" interna difícil de debugar.

Class loading

Class loading é o processo pelo qual classes são carregadas pela JVM.

Importante para entender:

- Problemas com dependências.
- Conflitos de versão.
- Classpath.
- Modularização.

- Frameworks.
- Containers.
- Hot reload.
- Erros como `ClassNotFoundException` e `NoClassDefFoundError`.
- Imutabilidade

Imutabilidade reduz efeitos colaterais e facilita concorrência.

Usado em:

- Value Objects.
- DTOs de entrada/saída.
- Configurações.
- Eventos.
- Commands.
- Objetos compartilhados entre threads.

Erro comum: criar objetos parcialmente mutáveis sem necessidade.

Concorrência, virtual threads e structured concurrency

Concorrência em Java envolve múltiplas tarefas sendo executadas de forma sobreposta.

Você precisa estudar:

- Threads tradicionais.
- Executors.
- `CompletableFuture`.
- Locks.
- Atomic classes.
- Concurrent collections.
- Thread safety.
- Memory Model.
- Virtual threads.
- Structured concurrency.
- Threads tradicionais

São boas para muitos cenários, mas caras em grande volume porque cada thread tradicional consome recursos significativos do sistema operacional.

Virtual threads

São threads leves gerenciadas pela JVM, úteis para workloads IO-bound, como chamadas HTTP, banco e integrações.

Não resolvem automaticamente:

- CPU-bound pesado.
- Locks mal utilizados.
- Pool de conexão pequeno.
- Banco lento.
- APIs externas instáveis.
- Reactive programming

Reativo é útil quando você precisa de alto throughput com modelo não bloqueante, mas aumenta complexidade cognitiva.

Critério sênior

Saber comparar:

- Thread tradicional: simples, mas limitada em grande escala.
- Reactive: eficiente, mas mais complexo.
- Virtual threads: simplificam concorrência bloqueante, mas exigem cuidado com recursos externos.
- JVM, Garbage Collection, JFR e JMC

A JVM gerencia memória, execução, compilação JIT e garbage collection.

Você precisa saber analisar:

- Heap.
- Stack.
- Metaspace.
- GC pauses.
- CPU.
- Threads bloqueadas.
- Deadlocks.
- Memory leaks.
- Allocation rate.
- Thread dump.
- Heap dump.

Ferramentas:

- JFR: Java Flight Recorder.
- JMC: Java Mission Control.
- VisualVM.
- Async profiler.
- Logs de GC.
- Onde é usado em projetos reais
- APIs de alta concorrência.
- Serviços de pedidos.
- Processamento assíncrono.
- Integrações com gateways.
- Jobs de importação.
- Microserviços com alto volume.
- Diagnóstico de incidentes.
- Redução de latência.
- Investigação de memory leak.
- Problemas que resolve
- Código verboso.
- NullPointerException.
- Baixa legibilidade.
- Concorrência insegura.
- Gargalos de CPU.

- Vazamento de memória.
- Alto tempo de resposta.
- Uso incorreto da JVM.
- Threads bloqueadas.
- GC excessivo.
- Erros comuns
- Usar Stream onde for seria mais claro.
- Usar Optional em DTO ou entidade.
- Ignorar thread safety.
- Achar que virtual threads resolvem tudo.
- Não configurar pool de banco.
- Não analisar heap/thread dump.
- Criar abstrações genéricas demais.
- Usar reflection sem necessidade.
- Confundir paralelismo com concorrência.
- Profundidade esperada de um sênior

Um profissional sênior precisa:

- Saber diagnosticar gargalos com evidência.
- Explicar impacto de CPU, IO, memória e GC.
- Escolher modelo de concorrência adequado.
- Escrever código idiomático, simples e testável.
- Revisar código considerando manutenibilidade.
- Entender trade-offs entre clareza, performance e abstração.

2. Spring Boot profissional

O que é

É o domínio do ecossistema Spring Boot 3 e Spring Framework 6 para criar aplicações backend robustas, seguras, observáveis e resilientes.

Spring Boot não é apenas criar controller, service e repository. Em nível profissional, envolve configuração, lifecycle, segurança, métricas, validação, resiliência, integração, cache e operação.

Para que serve

Serve para construir:

- APIs REST.
- Microserviços.
- Monólitos modulares.
- Workers.
- Consumers Kafka.
- Jobs agendados.
- Serviços de integração.
- Backends corporativos.
- Aplicações cloud-native.
- Principais conceitos

- Dependency Injection

DI permite que objetos recebam suas dependências de fora, reduzindo acoplamento.

Usado para:

- Services.
- Use cases.
- Repositories.
- Adapters.
- Clients externos.
- Strategies.

Erro comum: injetar dependências demais em uma classe, indicando violação de responsabilidade única.

Bean lifecycle

Entender o ciclo de vida dos beans permite controlar inicialização, configuração e destruição.

Importante para:

- Configurações customizadas.
- Clients externos.
- Recursos de infraestrutura.
- Inicialização de caches.
- Listeners.
- Observabilidade.
- Profiles e configuration properties

Profiles permitem separar configurações por ambiente:

- local
- dev
- staging
- prod
- test

Configuration properties permitem mapear configurações com tipagem forte.

Erro comum: deixar configuração crítica hardcoded no código.

Actuator, health checks e metrics

Actuator expõe endpoints operacionais.

Usado para:

- Health check.
- Readiness.
- Liveness.
- Métricas.
- Info da aplicação.
- Prometheus.
- Monitoramento.

Em produção, health check precisa validar dependências essenciais sem derrubar aplicação por falso positivo.

Validation

Bean Validation permite validar entrada de dados.

Usado em:

- DTOs de request.
- Commands.
- APIs públicas.
- Payloads de integração.

Erro comum: colocar validação de negócio complexa apenas em annotation. Validação estrutural fica no DTO; regra de negócio fica no domínio ou caso de uso.

Exception handling e RFC 7807

Tratamento de erro padronizado melhora integração com clientes.

RFC 7807 define um formato para detalhes de problema em APIs HTTP.

Um erro bem modelado deve conter:

- Tipo.
- Título.
- Status HTTP.
- Detalhe.
- Código de erro interno.
- Trace/correlation ID.
- Campos inválidos, quando aplicável.
- Filters e Interceptors

Usados para preocupações transversais:

- Logging.
- Correlação.
- Autenticação.
- Auditoria.
- Métricas.
- Headers.
- Rate limiting.

Erro comum: colocar regra de negócio em filter/interceptor.

Schedulers e async processing

Schedulers executam tarefas recorrentes. Async processing executa tarefas fora do fluxo principal.

Usado para:

- Reprocessamento.
- Limpeza de dados.
- Sincronizações.
- Envio de notificações.
- Jobs internos.

Cuidado: jobs precisam de idempotência, lock distribuído e observabilidade.

Cache abstraction

Spring Cache abstrai cache com Redis, Caffeine ou outros providers.

Usado para:

- Dados de leitura frequente.
- Configurações.
- Catálogos.
- Consultas caras.

Erro comum: cachear dado mutável sem estratégia de invalidação.

Feign, WebClient e integrações

Feign é declarativo e simples. WebClient é mais flexível e suporta não bloqueante.

Pontos críticos:

- Timeout.
- Retry.
- Circuit breaker.
- Fallback.
- Logs.
- Headers.
- Segurança.
- Observabilidade.
- Tratamento de erro.
- Resilience4j

Resilience4j implementa padrões de resiliência:

- Circuit breaker.
- Retry.
- Rate limiter.
- Bulkhead.
- Timeout.
- Fallback.

Critério sênior: saber configurar retry sem amplificar falhas. Retry mal configurado pode derrubar sistemas dependentes.

Onde é usado em projetos reais

- APIs de clientes.
- Sistemas de pedidos.
- Plataformas financeiras.
- Integrações com ERPs.
- Serviços de checkout.

- BFFs.
- Microserviços.
- Workers assíncronos.
- Gateways internos.
- Problemas que resolve
- Baixo acoplamento.
- Configuração por ambiente.
- Tratamento padronizado de erro.
- Observabilidade.
- Segurança.
- Integrações resilientes.
- Menor boilerplate.
- Padronização técnica.
- Erros comuns
- Criar services gigantes.
- Colocar regra de domínio em controller.
- Não configurar timeout.
- Retry infinito.
- Circuit breaker sem métrica.
- Expor actuator sem segurança.
- Retornar exceção bruta para cliente.
- Não diferenciar erro técnico de erro de negócio.
- Usar cache sem invalidação.
- Profundidade sênior

Um sênior precisa:

- Projetar aplicações Spring observáveis e resilientes.
- Definir padrões de exception handling.
- Configurar integração externa com timeout, retry e fallback.
- Saber quando usar Feign ou WebClient.
- Isolar infraestrutura de domínio.
- Padronizar logs, métricas e tracing.
- Criar APIs com contrato claro e versionável.

3. API Design avançado

O que é

API Design é a disciplina de projetar contratos de comunicação claros, consistentes, seguros e evolutivos.

Não é apenas criar endpoints. É definir como clientes e sistemas vão interagir com estabilidade ao longo do tempo.

Para que serve

Serve para:

- Reduzir quebra de contrato.
- Melhorar integração.

- Padronizar erros.
- Facilitar documentação.
- Permitir evolução sem breaking changes.
- Aumentar previsibilidade para frontend, mobile e parceiros.
- Conceitos principais
- REST maturity

O modelo de maturidade REST avalia o quanto uma API utiliza corretamente recursos HTTP.

Pontos importantes:

- Recursos bem nomeados.
- Verbos HTTP corretos.
- Status codes adequados.
- Representações consistentes.
- Links/HATEOAS quando fizer sentido.
- OpenAPI e Swagger

OpenAPI define o contrato formal da API. Swagger geralmente é usado para visualizar e testar esse contrato.

Deve conter:

- Endpoints.
- Requests.
- Responses.
- Headers.
- Status codes.
- Exemplos.
- Schemas.
- Erros.
- Autenticação.
- Versionamento.
- Idempotência

Uma operação idempotente pode ser repetida sem causar efeito duplicado.

Importante em:

- Pagamentos.
- Criação de pedidos.
- Retentativas.
- Integração com APIs externas.
- Mensageria.

Exemplo: usar Idempotency-Key para evitar duplicidade em uma operação crítica.

Paginação offset vs cursor

Offset é simples, mas pode ficar lento e inconsistente em grandes volumes.

Cursor é melhor para dados grandes e ordenação estável.

Use cursor quando:

- Volume é alto.
- Há scroll infinito.

- A lista muda frequentemente.
- Performance importa.
- Filtering e sorting

Devem ser previsíveis, documentados e seguros.

Evite permitir filtros arbitrários que exponham estrutura interna do banco ou gerem queries ruins.

Versionamento

Pode ser feito por:

- URL: /v1/orders
- Header.
- Media type.
- Estratégia de compatibilidade sem versão explícita.

O ponto sênior é definir política de evolução e depreciação.

HATEOAS conceitual

HATEOAS permite que a API indique links de próximas ações.

É útil em APIs públicas e altamente navegáveis, mas pode ser excessivo em APIs internas simples.

HTTP status codes

Saber usar corretamente:

- 200 OK
- 201 Created
- 202 Accepted
- 204 No Content
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 409 Conflict
- 422 Unprocessable Entity
- 429 Too Many Requests
- 500 Internal Server Error
- 503 Service Unavailable
- ETags e Cache-Control

ETags ajudam em cache e concorrência otimista HTTP.

Cache-Control define comportamento de cache em clientes e intermediários.

Usado em:

- Catálogos.
- Conteúdo público.
- Recursos pouco mutáveis.
- APIs de leitura intensiva.
- API Gateway e BFF

API Gateway centraliza entrada, segurança, roteamento, rate limiting e observabilidade.

BFF é um backend específico para uma interface, como web, mobile ou painel interno.

Erro comum: transformar API Gateway em lugar de regra de negócio.

Backward compatibility e deprecation strategy

APIs precisam evoluir sem quebrar clientes.

Estratégias:

- Adicionar campos sem remover antigos.
- Não mudar significado de campos existentes.
- Manter contrato antigo por janela definida.
- Documentar depreciação.
- Medir consumo antes de remover.
- Onde é usado em projetos reais
- APIs públicas.
- Integração com parceiros.
- Apps mobile.
- Frontend web.
- Microserviços internos.
- Plataformas SaaS.
- Gateways corporativos.
- Problemas que resolve
- Contratos ambíguos.
- Quebra de clientes.
- Erros inconsistentes.
- Falta de rastreabilidade.
- Dificuldade de integração.
- Evolução desorganizada.
- Erros comuns
- Usar sempre 200 para tudo.
- Retornar mensagens de erro sem padrão.
- Não documentar exemplos.
- Não pensar em idempotência.
- Versionar tarde demais.
- Criar endpoints orientados a tabelas, não a casos de uso.
- Expor detalhes internos do domínio.
- Profundidade sênior

Um sênior precisa:

- Justificar REST vs gRPC vs GraphQL vs eventos.
- Projetar contrato evolutivo.
- Documentar cenários de sucesso e erro.
- Definir política de versionamento.
- Considerar segurança, rate limit, cache e observabilidade.
- Evitar acoplamento entre cliente e modelo interno.

4. Persistência e banco com Java

O que é

É o domínio de acesso a dados em Java usando JPA/Hibernate, SQL, transações, otimização de queries, modelagem, migrações e padrões de consistência.

Para que serve

Serve para construir persistência:

- Correta.
- Performática.
- Consistente.
- Auditável.
- Evolutiva.
- Segura.
- Orientada ao caso de uso.
- Conceitos principais
- JPA/Hibernate avançado

JPA é a especificação. Hibernate é uma implementação.

Você precisa entender:

- Entity lifecycle.
- Persistence context.
- Dirty checking.
- Lazy/eager loading.
- Flush.
- Merge.
- Detach.
- Cascades.
- Proxies.
- Entity lifecycle

Estados principais:

- New/transient.
- Managed.
- Detached.
- Removed.

Isso afeta persistência, atualização e comportamento transacional.

Persistence context e dirty checking

Persistence context mantém entidades gerenciadas durante a transação.

Dirty checking detecta alterações e sincroniza com o banco.

Erro comum: alterar entidade sem entender quando o flush acontece.

Lazy/eager loading e N+1

Lazy carrega sob demanda. Eager carrega imediatamente.

N+1 ocorre quando uma consulta inicial gera várias consultas adicionais.

Soluções:

- Fetch join.
- EntityGraph.
- Batch size.
- DTO projection.
- Query específica por caso de uso.
- Locks otimistas e pessimistas

Optimistic locking usa versão para evitar atualização concorrente perdida.

Pessimistic locking bloqueia registros no banco.

Use optimistic para maioria dos casos. Use pessimistic quando concorrência crítica exigir bloqueio forte.

Transações, isolation levels e propagation

Transações garantem atomicidade.

Isolation levels controlam visibilidade entre transações:

- Read uncommitted.
- Read committed.
- Repeatable read.
- Serializable.

Propagation define como métodos transacionais se comportam quando já existe transação.

Critério sênior: saber explicar impacto real de isolamento em concorrência, consistência e performance.

Flyway

Gerencia migrações versionadas de banco.

Usado para:

- Criar tabelas.
- Alterar schema.
- Versionar evolução.
- Manter ambientes consistentes.
- HikariCP

Pool de conexões padrão muito usado no Spring Boot.

Pontos críticos:

- Maximum pool size.
- Connection timeout.
- Idle timeout.
- Lifetime.
- Métricas.
- Relação com threads da aplicação.

Erro comum: aumentar pool sem entender limite do banco.

Query tuning e índices

Tuning envolve analisar:

- Plano de execução.
- Índices.
- Cardinalidade.
- Joins.
- Paginação.
- Locks.
- Tempo de resposta.
- Leitura em disco.
- Soft delete, auditoria e multi-tenancy

Soft delete marca registro como removido sem apagar fisicamente.

Auditoria rastreia criação, alteração e usuário.

Multi-tenancy separa dados de diferentes clientes.

Estratégias de multi-tenancy:

- Coluna tenant_id.
- Schema por tenant.
- Banco por tenant.
- Outbox pattern e CDC com Debezium

Outbox resolve consistência entre banco e mensageria.

Fluxo:

- Transação grava dado de negócio.
- Na mesma transação grava evento em tabela outbox.
- Processo publica evento no broker.
- Consumidores processam.

CDC com Debezium captura alterações no banco e publica em eventos.

Onde é usado em projetos reais

- Sistemas financeiros.
- Pedidos.
- Estoque.
- Faturamento.
- CRM.
- ERP.
- Auditoria.
- Integrações críticas.
- Microserviços com eventos.
- Problemas que resolve
- N+1.
- Inconsistência transacional.
- Lentidão em consultas.
- Duplicidade.

- Perda de evento.
- Concorrência incorreta.
- Evolução desorganizada do schema.
- Pool saturado.
- Erros comuns
- Usar entidade como DTO.
- Criar relacionamentos eager demais.
- Não entender transação.
- Não analisar query gerada.
- Ignorar índice.
- Pagar com offset em tabela enorme.
- Usar cascade sem critério.
- Ajustar pool de conexão no chute.
- Publicar evento fora de uma estratégia transacional.
- Profundidade sênior

Um sênior precisa:

- Modelar persistência por caso de uso.
- Resolver N+1 sem "gambiarras".
- Explicar isolamento transacional.
- Ajustar pool com base em métricas.
- Definir estratégia de migração.
- Usar outbox quando há consistência entre banco e eventos.
- Revisar queries por impacto em produção.

5. Arquitetura de software

O que é

É a disciplina de organizar software em componentes, camadas, módulos, fronteiras e contratos para controlar complexidade, evolução, testabilidade e acoplamento.

Para que serve

Serve para:

- Reduzir acoplamento.
- Aumentar coesão.
- Facilitar testes.
- Proteger domínio.
- Permitir evolução.
- Organizar times.
- Documentar decisões.
- Evitar caos estrutural.
- Conceitos principais
- Clean Architecture

Separa o software em camadas com dependências apontando para o domínio.

Ideia central:

- Domínio não depende de framework.
- Casos de uso orquestram regras.
- Infraestrutura implementa detalhes.
- Interface entrega entrada e saída.
- Hexagonal Architecture

Também chamada Ports and Adapters.

Elementos:

- Domínio.
- Application/use cases.
- Ports de entrada.
- Ports de saída.
- Adapters REST, Kafka, banco, APIs externas.

É excelente para Java/Spring porque permite isolar Spring e infraestrutura do núcleo da aplicação.

DDD

Domain-Driven Design foca em modelar software a partir do domínio de negócio.

Conceitos:

- Bounded Context.
- Aggregates.
- Entities.
- Value Objects.
- Domain Events.
- Repositories.
- Services.
- Ubiquitous Language.
- Bounded Context

Define uma fronteira onde determinado modelo tem significado específico.

Exemplo:

- Pedido no contexto de vendas.
- Pedido no contexto de logística.
- Pedido no contexto financeiro.

O mesmo termo pode ter significados diferentes em contextos diferentes.

Aggregates

Um aggregate é um conjunto de objetos tratado como unidade de consistência.

Exemplo:

- Order como aggregate root.
- OrderItem dentro do pedido.
- Regras de alteração passam pelo root.

Erro comum: criar aggregate enorme que tenta representar o banco inteiro.

Value Objects

Objetos sem identidade própria, comparados por valor.

Exemplos:

- Money.
- Email.
- CPF.
- Address.
- Period.
- Quantity.

Value Objects reduzem duplicação de validação e tornam regras explícitas.

Domain Events

Representam algo relevante que aconteceu no domínio.

Exemplos:

- OrderCreated
- PaymentApproved
- InvoiceIssued
- CustomerRegistered

Não são apenas mensagens técnicas. São fatos do domínio.

Modular Monolith

Monólito modular organiza o sistema em módulos bem separados, com fronteiras explícitas.

É muitas vezes melhor do que microserviços no início, porque reduz complexidade operacional.

Microservices

Microserviços são serviços independentes, com deploy e dados próprios.

Usar quando há necessidade real de:

- Escala independente.
- Autonomia de times.
- Fronteiras de domínio maduras.
- Resiliência organizacional.
- Evolução isolada.

Erro comum: criar microserviços sem maturidade de observabilidade, CI/CD, testes, ownership e arquitetura de dados.

Event-driven architecture

Arquitetura orientada a eventos permite comunicação assíncrona entre sistemas.

Resolve:

- Baixo acoplamento temporal.
- Processos longos.
- Integrações distribuídas.
- Escalabilidade de consumidores.

Mas cria desafios:

- Consistência eventual.
- Idempotência.
- Observabilidade.
- Reprocessamento.
- Ordenação.
- CQRS

Separa modelo de escrita e leitura.

Útil quando:

- Leitura e escrita têm necessidades diferentes.
- Consultas são complexas.
- Escala de leitura é maior.
- Projeções são necessárias.
- Saga

Coordena transações distribuídas por etapas.

Pode ser:

- Coreografada por eventos.
- Orquestrada por um coordenador.

Usado em pedidos, pagamento, estoque, entrega e cancelamento.

ADRs e C4 Model

ADR documenta decisão arquitetural.

C4 Model documenta arquitetura em níveis:

- Contexto.
- Container.
- Component.
- Code.
- Onde é usado em projetos reais
- Sistemas enterprise.
- Plataformas de pagamento.
- E-commerce.
- Sistemas de pedido.
- Backoffice.
- ERPs.
- CRMs.
- SaaS multi-tenant.
- Microserviços com eventos.
- Problemas que resolve
- Código acoplado ao framework.
- Domínio espalhado.
- Baixa testabilidade.
- Entidades anêmicas.
- Microserviços desnecessários.

- Falta de documentação.
- Evolução arriscada.
- Duplicidade de regras.
- Erros comuns
- Confundir DDD com criar muitas pastas.
- Criar arquitetura hexagonal burocrática demais.
- Usar microserviços sem necessidade.
- Criar use case que só chama repository.
- Colocar regra de negócio em controller.
- Criar domínio dependente de Spring.
- Não documentar trade-offs.
- Profundidade sênior

Um sênior precisa:

- Escolher arquitetura pelo problema, não por moda.
- Defender monólito modular quando fizer sentido.
- Definir fronteiras de domínio.
- Separar aplicação, domínio, infraestrutura e interface.
- Documentar decisões com trade-offs.
- Evoluir arquitetura sem reescrever tudo.
- Balancear pureza arquitetural e entrega de valor.

6. System Design avançado

O que é

System Design é a capacidade de projetar sistemas distribuídos considerando escala, latência, consistência, disponibilidade, custo, operação, falhas e evolução.

Para que serve

Serve para desenhar sistemas que suportam:

- Alto volume.
- Crescimento.
- Falhas parciais.
- Baixa latência.
- Alta disponibilidade.
- Recuperação de desastre.
- Evolução contínua.
- Conceitos principais
- Escalabilidade vertical vs horizontal

Vertical: aumentar recursos da máquina.

Horizontal: adicionar mais máquinas/instâncias.

Sênior precisa saber quando cada uma é suficiente e quando o custo/limite exige mudança.

Load balancing

Distribui tráfego entre instâncias.

Importante para:

- Alta disponibilidade.
- Escalabilidade.
- Failover.
- Blue/green deployment.
- Rolling update.
- CDN

Distribui conteúdo perto do usuário.

Usado para:

- Imagens.
- Vídeos.
- Arquivos estáticos.
- Conteúdo público.
- Redução de latência global.
- Cache distribuído

Cache com Redis ou similar reduz carga no banco e melhora latência.

Padrões:

- Cache aside.
- Write-through.
- Write-behind.
- Cache invalidation.
- Proteção contra cache stampede.

Erro comum: cachear tudo sem estratégia de expiração e invalidação.

Sharding e consistent hashing

Sharding divide dados entre partições.

Consistent hashing ajuda a distribuir chaves minimizando remanejamento quando nós entram ou saem.

Usado em:

- Bancos distribuídos.
- Caches.
- Sistemas de armazenamento.
- Filas particionadas.
- Replicação e quorum

Replicação copia dados entre nós.

Modelos:

- Leader/follower.
- Multi-leader.
- Quorum.

Problemas:

- Replication lag.

- Conflitos.
- Escritas inconsistentes.
- Leitura stale.
- CAP e PACELC

CAP trata consistência, disponibilidade e tolerância a partição.

PACELC complementa analisando trade-off entre latência e consistência mesmo quando não há partição.

Sênior precisa aplicar isso em decisões reais, não apenas decorar teoria.

Consensus, Raft, Paxos e leader election

Consenso é necessário para vários nós concordarem sobre estado.

Usado em:

- Coordenação distribuída.
- Bancos distribuídos.
- Sistemas com leader election.
- Controle de cluster.
- Split-brain, clock skew e vector clocks

Split-brain ocorre quando partes do cluster acreditam ser líderes.

Clock skew é diferença de relógio entre máquinas.

Vector clocks ajudam a detectar causalidade e conflitos em sistemas distribuídos.

CRDTs

Estruturas que convergem automaticamente em sistemas distribuídos.

Usadas em colaboração em tempo real, sistemas offline-first e replicação eventual.

Multi-region, disaster recovery, RPO e RTO

Multi-região melhora disponibilidade global, mas aumenta complexidade.

RPO: quanto dado pode ser perdido.

RTO: quanto tempo o sistema pode ficar indisponível.

Fan-out on write vs fan-out on read

Usado em feeds.

Fan-out on write: pré-distribui conteúdo quando publicado.

Fan-out on read: monta feed quando usuário lê.

Trade-off entre escrita, leitura, armazenamento e latência.

WebSocket, SSE e push notifications

Usados para comunicação em tempo real.

- WebSocket: bidirecional.
- SSE: servidor envia eventos para cliente.
- Push: notificação fora do app.
- Backpressure

Backpressure impede que produtores sobrecarreguem consumidores.

Essencial em:

- Streams.
- Filas.
- APIs sob carga.
- Sistemas reativos.
- Processamento assíncrono.
- Projetos práticos recomendados
- Sistema de notificações em escala.
- Encurtador de URL com cache e rate limit.
- Sistema de feed.
- Sistema de pedidos com pagamento assíncrono.
- Plataforma de conteúdo multi-região.
- Profundidade sênior

Um sênior precisa:

- Desenhar arquitetura sob requisitos concretos.
- Explicar trade-offs de consistência e disponibilidade.
- Estimar gargalos.
- Definir cache, banco, filas e replicação.
- Planejar falhas.
- Considerar custo operacional.
- Projetar observabilidade desde o início.

7. gRPC e Protocol Buffers

O que é

gRPC é um framework de comunicação RPC de alta performance baseado em HTTP/2. Protocol Buffers é o mecanismo de serialização e contrato.

Para que serve

Serve para comunicação eficiente entre serviços, especialmente em ambientes internos de microserviços.

Usado em:

- Comunicação service-to-service.
- Baixa latência.
- Contratos fortemente tipados.
- Streaming.
- Sistemas com alto volume.
- Plataformas internas.
- Conceitos principais
- .proto

Arquivo que define contratos:

- Messages.
- Enums.
- Services.
- Métodos.
- Tipos.

- Campos.
- Unary e streaming

Tipos de chamada:

- Unary: uma request, uma response.
- Server streaming: uma request, várias responses.
- Client streaming: várias requests, uma response.
- Bidirectional streaming: fluxo nos dois sentidos.
- Status codes, deadlines e retries

gRPC tem seus próprios status codes.

Deadlines são essenciais para evitar chamadas penduradas.

Retry deve ser usado com cuidado, considerando idempotência.

Interceptors, auth e tracing

Interceptors permitem cross-cutting concerns:

- Autenticação.
- Autorização.
- Logs.
- Métricas.
- Tracing.
- Correlation ID.
- Backward compatibility e schema evolution

Protobuf exige cuidado com evolução:

- Não reutilizar número de campo removido.
- Não mudar significado de campo.
- Manter compatibilidade.
- Marcar campos obsoletos.
- Planejar evolução de contrato.
- REST vs gRPC

Use REST quando:

- API é pública.
- Precisa ser facilmente consumida por browsers.
- Simplicidade é prioridade.
- Ecossistema HTTP padrão é suficiente.

Use gRPC quando:

- Comunicação é interna.
- Performance importa.
- Contrato forte importa.
- Streaming é necessário.
- Microserviços precisam de comunicação eficiente.
- Erros comuns
- Usar gRPC para API pública sem necessidade.
- Ignorar deadlines.

- Não tratar erro adequadamente.
- Quebrar contrato Protobuf.
- Não configurar tracing.
- Comparar gRPC e REST apenas por velocidade.
- Profundidade sênior

Um sênior precisa saber:

- Quando gRPC compensa.
- Quando REST é suficiente.
- Como evoluir contratos Protobuf.
- Como tratar erro, timeout e observabilidade.
- Como comparar latência, custo e complexidade.

8. Kafka avançado, Kafka Streams e Flink

O que é

Kafka é uma plataforma de streaming de eventos. Kafka Streams é uma biblioteca para processamento de stream em aplicações Java. Flink é um engine avançado de stream processing stateful.

Para que serve

Serve para:

- Comunicação assíncrona.
- Event-driven architecture.
- Processamento de eventos.
- Integração entre sistemas.
- Pipelines de dados.
- Auditoria.
- Reprocessamento.
- Desacoplamento.
- Conceitos principais
- Producers, consumers e consumer groups

Producer publica eventos. Consumer consome. Consumer group permite dividir carga entre instâncias.

Partitions e offset management

Partitions permitem paralelismo e ordenação por chave.

Offset indica posição de leitura.

Sênior precisa entender:

- Commit automático vs manual.
- Reprocessamento.
- Ordenação.
- Paralelismo.
- Lag.
- Rebalance

Rebalance redistribui partições entre consumidores.

Pode gerar pausa no consumo.

Causas:

- Nova instância.
- Instância caiu.
- Timeout.
- Configuração ruim.
- Processamento lento.
- DLQ, retry topics e poison messages

DLQ armazena mensagens que não puderam ser processadas.

Retry topics permitem retentativas controladas.

Poison message é uma mensagem que sempre falha.

Idempotent producer e transactions

Idempotent producer reduz duplicidade por falha de envio.

Transactions permitem atomicidade entre produção e consumo em certos cenários.

Exactly-once depende de configuração correta e não elimina a necessidade de idempotência no consumidor.

Schema Registry

Controla contratos de eventos.

Formatos:

- Avro.
- Protobuf.
- JSON Schema.

Ajuda a evitar quebra entre produtores e consumidores.

Kafka Streams

Permite processamento de stream em Java.

Conceitos:

- KStream.
- KTable.
- GlobalKTable.
- Windowing.
- Aggregations.
- Joins.
- State stores.
- Interactive queries.
- Flink conceitual

Flink é usado para processamento stateful avançado.

Conceitos:

- Checkpointing.
- State backend.
- Event time.
- Watermarks.
- Backpressure.

- Fault tolerance.
- Projeto prático: pipeline de pedidos

Eventos:

- order-created
- payment-approved
- invoice-issued
- order-completed

O projeto deve incluir:

- Schema Registry.
- DLQ.
- Retry topic.
- Idempotência.
- Correlation ID.
- Monitoramento de lag.
- Kafka Streams para agregação.
- Reprocessamento controlado.
- Erros comuns
- Não definir chave correta.
- Ignorar ordenação por partição.
- Não tratar duplicidade.
- Usar retry infinito.
- Não ter DLQ.
- Não monitorar lag.
- Criar evento sem contrato.
- Confundir evento de domínio com comando técnico.
- Não planejar reprocessamento.
- Profundidade sênior

Um sênior precisa:

- Projetar fluxo resiliente.
- Definir estratégia de idempotência.
- Garantir rastreabilidade.
- Planejar DLQ e reprocessamento.
- Controlar evolução de schemas.
- Monitorar lag e throughput.
- Saber quando Kafka não é necessário.

9. Testes e qualidade

O que é

É a capacidade de definir e implementar uma estratégia de qualidade que cubra unidade, integração, contrato, arquitetura, segurança, carga e regressão.

Para que serve

Serve para:

- Reduzir bugs.
- Proteger regra de negócio.
- Aumentar confiança em deploy.
- Evitar regressões.
- Validar contratos.
- Melhorar design.
- Sustentar refatorações.
- Ferramentas e práticas
- JUnit 5, Mockito e AssertJ

Base para testes unitários.

Usado para:

- Regras de domínio.
- Use cases.
- Services.
- Mappers.
- Validações.
- Tratamento de erro.
- MockMvc, WebTestClient e RestAssured

Usados para testar APIs.

- MockMvc: MVC tradicional.
- WebTestClient: web/reactive.
- RestAssured: testes HTTP mais próximos do uso real.
- Testcontainers

Sobe dependências reais em containers para teste.

Usado com:

- PostgreSQL.
- Kafka.
- Redis.
- MongoDB.
- RabbitMQ.
- WireMock

Simula APIs externas.

Essencial para testar:

- Timeout.
- Erro 500.
- Erro 404.
- Latência.
- Payload inesperado.
- Retry.
- Circuit breaker.
- Pact e contract testing

Garante compatibilidade entre consumidor e provedor.

Importante em microserviços e APIs internas.

Mutation testing

Avalia se os testes realmente detectam mudanças no código.

Ferramentas como PIT ajudam a medir qualidade real dos testes.

ArchUnit

Testa regras arquiteturais.

Exemplo:

- Domínio não pode depender de Spring.
- Controller não pode acessar repository diretamente.
- Use case não pode depender de adapter.
- Gatling e k6

Testes de carga e performance.

Usados para medir:

- Latência.
- Throughput.
- Erros.
- Saturação.
- Comportamento sob pico.
- SonarQube e JaCoCo

JaCoCo mede cobertura.

SonarQube analisa qualidade, bugs, code smells e vulnerabilidades.

Critério sênior: cobertura alta não significa qualidade se os testes não validam comportamento importante.

Erros comuns

- Testar implementação, não comportamento.
- Mockar tudo.
- Não testar integração com banco.
- Não testar contrato.
- Criar testes frágeis.
- Ignorar cenários de erro.
- Perseguir cobertura sem critério.
- Não testar concorrência/idempotência.
- Não rodar testes de carga.
- Profundidade sênior

Um sênior precisa:

- Definir estratégia de testes por risco.
- Saber o que testar em unidade, integração e contrato.
- Garantir teste para regra crítica.
- Criar testes que sustentam refatoração.
- Usar Testcontainers para infraestrutura real.

- Validar arquitetura com ArchUnit.
- Medir performance com k6/Gatling.

10. Soft skills de engenharia sênior

O que é

São competências de influência técnica, comunicação, revisão, mentoria, documentação e tomada de decisão.

Um sênior não é apenas quem sabe mais código. É quem melhora o sistema, o time e a qualidade das decisões.

Principais práticas

Code review efetivo

Um bom code review avalia:

- Clareza.
- Testabilidade.
- Segurança.
- Performance.
- Design.
- Consistência.
- Contratos.
- Tratamento de erro.
- Observabilidade.

Não deve ser apenas revisão de estilo.

Mentoria técnica

Mentoria envolve:

- Explicar conceitos.
- Dar feedback.
- Ajudar na evolução.
- Criar padrões.
- Compartilhar contexto.
- Formar autonomia.
- RFC técnico

RFC propõe mudanças relevantes antes da implementação.

Deve conter:

- Problema.
- Contexto.
- Opções.
- Trade-offs.
- Impactos.
- Riscos.
- Plano de rollout.
- Plano de rollback.

- ADR

ADR registra uma decisão arquitetural.

Deve conter:

- Contexto.
- Decisão.
- Consequências.
- Alternativas consideradas.
- Motivo da escolha.
- Postmortem

Postmortem analisa incidente sem foco em culpa.

Deve responder:

- O que aconteceu?
- Qual foi o impacto?
- Qual foi a causa?
- Como detectamos?
- Como mitigamos?
- Como evitar recorrência?
- Incident command

Capacidade de coordenar resposta a incidente:

- Priorizar mitigação.
- Organizar comunicação.
- Delegar investigação.
- Controlar ruído.
- Registrar timeline.
- Decidir rollback ou hotfix.
- Entregáveis práticos
- 10 ADRs.
- 5 RFCs.
- 5 postmortems simulados.
- Checklist de code review.
- Guia de onboarding técnico.
- Profundidade sênior

Um sênior precisa:

- Influenciar sem depender de cargo.
- Comunicar risco técnico para negócio.
- Negociar escopo.
- Priorizar dívida técnica.
- Orientar decisões de arquitetura.
- Ajudar outros devs a crescer.
- Criar documentação útil, não burocrática.

3. Plano de estudos em 6 módulos

Módulo 1 - Fundamentos avançados de Java e JVM

Objetivo

Construir base sólida em Java moderno, orientação a objetos, recursos da linguagem, concorrência e diagnóstico de JVM.

Conteúdos

- Java 17/21.
- Records.
- Sealed classes.
- Pattern matching.
- Switch expressions.
- Optional.
- Streams.
- Generics.
- Imutabilidade.
- Concorrência.
- Virtual threads.
- CompletableFuture.
- JVM, GC, JFR, JMC.
- Ordem recomendada
- Revisar Java Core e OO.
- Estudar recursos modernos.
- Aprofundar generics e collections.
- Estudar concorrência.
- Estudar JVM e profiling.
- Criar pequenos benchmarks e diagnósticos.
- Práticas recomendadas
- Refatorar DTOs para records.
- Criar Value Objects imutáveis.
- Implementar serviços com CompletableFuture.
- Comparar thread tradicional com virtual threads.
- Gerar heap dump e thread dump.
- Analisar execução com JFR.
- Exercícios técnicos
- Criar um coletor customizado com Collector.
- Criar uma hierarquia de erros com sealed classes.
- Simular 10 mil chamadas IO-bound com virtual threads.
- Criar um memory leak proposital e diagnosticar.
- Comparar performance de loops, streams e parallel streams.
- Mini projeto

Java Runtime Lab

Um projeto Java puro com:

- Processamento concorrente.
- Uso de records e sealed classes.
- Simulação de IO.
- Métricas simples.
- Análise com JFR.
- Testes unitários.
- Critérios de conclusão

Você concluiu este módulo quando consegue:

- Explicar recursos modernos do Java.
- Escolher entre thread tradicional, virtual thread e async.
- Diagnosticar CPU, memória e threads.
- Escrever código limpo e idiomático.
- Evitar uso inadequado de Optional, Stream e reflection.
- Nível esperado

Intermediário forte caminhando para avançado em Java.

Módulo 2 - Spring Boot profissional e APIs REST

Objetivo

Criar APIs profissionais com Spring Boot, boas práticas, validação, erros padronizados, segurança inicial, observabilidade e integrações resilientes.

Conteúdos

- Spring Boot 3.
- Spring Framework 6.
- Dependency Injection.
- Bean lifecycle.
- Profiles.
- Configuration properties.
- Controllers.
- Validation.
- Exception handling.
- Problem Details.
- Filters e interceptors.
- Actuator.
- Metrics.
- Feign/WebClient.
- Resilience4j.
- Spring Security básico/intermediário.
- Ordem recomendada
- Estrutura de aplicação Spring.
- Controllers e DTOs.
- Validação.

- Exception handling.
- Configuração por ambiente.
- Observabilidade.
- Integrações externas.
- Resiliência.
- Segurança.
- Práticas recomendadas
- Criar padrão de response de erro.
- Criar correlation ID.
- Configurar logs estruturados.
- Configurar health checks.
- Criar client externo com timeout.
- Aplicar retry apenas em erro transitório.
- Usar circuit breaker com fallback controlado.
- Exercícios técnicos
- Criar API REST de pedidos.
- Implementar paginação, filtros e ordenação.
- Criar tratamento de erro com Problem Details.
- Criar integração externa simulada com WireMock.
- Configurar timeout, retry e circuit breaker.
- Criar endpoint protegido com JWT.
- Mini projeto

Order Management API

API de pedidos com:

- CRUD orientado a caso de uso.
- Validação.
- Erros padronizados.
- OpenAPI.
- Logs com correlation ID.
- Actuator.
- Métricas.
- Integração externa simulada.
- Resilience4j.
- Testes.
- Critérios de conclusão

Você concluiu quando consegue:

- Criar API com contrato claro.
- Tratar erros de forma padronizada.
- Configurar resiliência sem efeito cascata.
- Diferenciar erro de negócio, técnico, transitório e integração.
- Operar minimamente a aplicação com métricas e health check.
- Nível esperado

Intermediário/avançado em Spring Boot.

Módulo 3 - Persistência, transações e qualidade

Objetivo

Dominar persistência com JPA/Hibernate, SQL, transações, migrations, tuning, testes de integração e estratégia de qualidade.

Conteúdos

- JPA/Hibernate avançado.
- Entity lifecycle.
- Persistence context.
- Dirty checking.
- Lazy/eager loading.
- N+1.
- EntityGraph.
- Fetch join.
- Locks.
- Transações.
- Isolation levels.
- Propagation.
- Flyway.
- HikariCP.
- Query tuning.
- Índices.
- Testcontainers.
- JaCoCo.
- SonarQube.
- ArchUnit.
- Ordem recomendada
- Modelagem relacional.
- Entidades e lifecycle.
- Consultas e loading.
- Transações.
- Locks.
- Performance.
- Migrations.
- Testes de integração.
- Testes arquiteturais.
- Práticas recomendadas
- Criar queries orientadas a caso de uso.
- Evitar expor entidade como response.
- Medir SQL gerado.
- Criar migrations versionadas.
- Testar repository com banco real via Testcontainers.
- Criar regras arquiteturais com ArchUnit.
- Exercícios técnicos

- Corrigir problema N+1.
- Criar consulta paginada performática.
- Simular concorrência com optimistic locking.
- Ajustar pool HikariCP sob carga.
- Criar teste de integração com PostgreSQL real.
- Mini projeto

Billing Persistence Lab

Sistema de faturamento com:

- Entidades.
- Value Objects.
- Migrations Flyway.
- Consultas otimizadas.
- Auditoria.
- Soft delete.
- Lock otimista.
- Testcontainers.
- Testes de integração.
- Critérios de conclusão

Você concluiu quando consegue:

- Resolver N+1.
- Explicar isolation levels.
- Usar transaction propagation corretamente.
- Criar migrations seguras.
- Otimizar consultas.
- Testar persistência com banco real.
- Justificar modelagem por caso de uso.
- Nível esperado

Avançado em persistência Java.

Módulo 4 - Arquitetura, DDD e modularização

Objetivo

Aprender a organizar sistemas com fronteiras claras, domínio protegido, baixo acoplamento e decisões arquiteturais documentadas.

Conteúdos

- Clean Architecture.
- Hexagonal Architecture.
- DDD.
- Bounded Context.
- Aggregates.
- Value Objects.
- Domain Events.
- Application Services.

- Use Cases.
- Ports and Adapters.
- Modular Monolith.
- Microservices.
- CQRS.
- Saga.
- Outbox.
- Inbox.
- Anti-corruption layer.
- ADR.
- C4 Model.
- Ordem recomendada
- Separação de camadas.
- Hexagonal Architecture.
- DDD tático.
- DDD estratégico.
- Modular monolith.
- Eventos de domínio.
- Outbox/inbox.
- Microserviços.
- ADR e C4.
- Práticas recomendadas
- Criar domínio sem dependência de Spring.
- Usar ports para integrações externas.
- Separar adapters REST, banco e mensageria.
- Documentar decisões com ADR.
- Desenhar arquitetura com C4.
- Começar com monólito modular antes de quebrar em serviços.
- Exercícios técnicos
- Refatorar API em arquitetura hexagonal.
- Criar aggregate de pedido.
- Criar Value Object Money.
- Criar evento OrderCreated.
- Criar camada de application/use case.
- Criar ADR comparando monólito modular vs microserviços.
- Mini projeto

Modular Commerce Platform

Plataforma modular com módulos:

- Catalog.
- Orders.
- Payments.
- Customers.
- Notifications.

Com:

- Arquitetura hexagonal.
- DDD.
- Domain events.
- Outbox.
- C4.
- ADRs.
- Critérios de conclusão

Você concluiu quando consegue:

- Separar domínio, aplicação, infraestrutura e interface.
- Definir bounded contexts.
- Escolher modular monolith ou microservices com justificativa.
- Documentar trade-offs.
- Evitar domínio anêmico.
- Criar módulos independentes e testáveis.
- Nível esperado

Avançado arquitetural.

Módulo 5 - Produção, performance, segurança, mensageria e escala

Objetivo

Preparar o estudante para projetar e operar sistemas reais em produção, com resiliência, observabilidade, mensageria, gRPC, Kafka, performance e system design.

Conteúdos

- Observabilidade.
- Logs.
- Metrics.
- Tracing.
- Kafka avançado.
- Kafka Streams.
- Schema Registry.
- DLQ.
- Retry topics.
- Idempotência.
- gRPC.
- Protobuf.
- System Design.
- Cache.
- Rate limiting.
- Backpressure.
- Load balancing.
- Multi-region.
- Disaster recovery.

- Segurança em APIs.
- Ordem recomendada
- Observabilidade.
- Resiliência.
- Kafka básico/intermediário.
- Kafka avançado.
- gRPC.
- Cache e performance.
- System Design.
- Segurança.
- Deploy e operação.
- Práticas recomendadas
- Criar eventos versionados.
- Usar Schema Registry.
- Configurar DLQ.
- Monitorar consumer lag.
- Criar dashboards.
- Testar carga com k6.
- Implementar rate limiting.
- Criar tracing distribuído.
- Simular falhas de dependências.
- Exercícios técnicos
- Criar consumer idempotente.
- Simular poison message.
- Criar DLQ e reprocessamento.
- Comparar REST vs gRPC.
- Criar cache aside com Redis.
- Criar teste de carga.
- Simular queda de serviço externo.
- Mini projeto

Async Order Platform

Sistema com:

- API de pedidos.
- Kafka.
- Serviço de pagamento.
- Serviço de nota fiscal.
- Serviço de notificação.
- Outbox.
- DLQ.
- Reprocessamento.
- Observabilidade.
- Teste de carga.
- Cache.

- Rate limit.
- Critérios de conclusão

Você concluiu quando consegue:

- Projetar fluxo assíncrono resiliente.
- Definir idempotência.
- Monitorar lag.
- Tratar falhas.
- Projetar cache.
- Explicar trade-offs de consistência.
- Comparar REST, gRPC e eventos.
- Diagnosticar gargalos em produção.
- Nível esperado

Avançado/sênior técnico.

Módulo 6 - Domínio sênior e liderança técnica

Objetivo

Consolidar domínio técnico, comunicação, documentação, revisão, mentoria, estratégia e tomada de decisão.

Conteúdos

- Code review.
- Mentoria.
- RFC.
- ADR.
- Postmortem.
- Incident command.
- Comunicação com negócio.
- Estimativas.
- Priorização técnica.
- Gestão de débito técnico.
- Documentação para times remotos.
- Ordem recomendada
- Code review.
- ADR.
- RFC.
- Postmortem.
- Incident response.
- Mentoria.
- Comunicação executiva.
- Gestão de dívida técnica.
- Práticas recomendadas
- Revisar PRs com checklist.
- Escrever ADRs reais.
- Criar RFC para mudança arquitetural.
- Simular incidente.

- Criar guia de onboarding.
- Apresentar trade-offs para pessoas técnicas e não técnicas.
- Exercícios técnicos
- Escrever 10 ADRs.
- Escrever 5 RFCs.
- Criar 5 postmortems simulados.
- Criar checklist de code review.
- Criar documentação de arquitetura C4.
- Conduzir uma apresentação técnica de 30 minutos.
- Mini projeto

Engineering Leadership Pack

Pacote com:

- ADRs.
- RFCs.
- Postmortems.
- Checklist de review.
- Guia de onboarding.
- Diagrama C4.
- Estratégia de testes.
- Estratégia de deploy.
- Plano de evolução técnica.
- Critérios de conclusão

Você concluiu quando consegue:

- Explicar decisões com trade-offs.
- Influenciar tecnicamente sem autoridade formal.
- Revisar código com profundidade.
- Comunicar risco técnico ao negócio.
- Conduzir resposta a incidente.
- Documentar arquitetura de forma útil.
- Mentorar devs menos experientes.
- Nível esperado

Sênior em comportamento técnico e influência.

4. Critérios de domínio técnico por grande área

Java e JVM

Nível Domínio esperado

Iniciante Entende sintaxe, OO, collections, exceptions e tipos básicos.

Intermediário Usa Java moderno, streams, Optional, generics e testes unitários.

Avançado Domina concorrência, JVM, GC, profiling, memória e performance.

Sênior Decide modelo de concorrência, diagnostica produção e orienta padrões de código.

Spring Boot e APIs

Nível Domínio esperado

Iniciante Cria controllers, services e repositories básicos.

Intermediário Implementa validação, exceptions, DTOs, OpenAPI e segurança básica.

Avançado Configura observabilidade, resiliência, cache, integração e segurança robusta.

Sênior Define padrões de API, contratos, versionamento, erros e operação em produção.

Persistência

Nível Domínio esperado

Iniciante Faz CRUD com JPA e SQL básico.

Intermediário Usa relacionamentos, queries, migrations e transações simples.

Avançado Resolve N+1, tuning, locks, índices, pool e transações complexas.

Sênior Modela persistência por caso de uso, define consistência e revisa impacto em produção.

Arquitetura

Nível Domínio esperado

Iniciante Entende camadas básicas.

Intermediário Aplica services, DTOs e separação de responsabilidades.

Avançado Aplica hexagonal, DDD, modularização, eventos e ADRs.

Sênior Define fronteiras, decide monólito/microserviços e documenta trade-offs.

System Design

Nível Domínio esperado

Iniciante Entende HTTP, banco, cache e filas conceitualmente.

Intermediário Desenha sistemas simples com API, banco, cache e mensageria.

Avançado Considera escala, disponibilidade, consistência, latência e operação.

Sênior Projeta sistemas distribuídos sob requisitos reais e justifica custos e riscos.

Kafka, gRPC e comunicação

Nível Domínio esperado

Iniciante Entende producer, consumer e REST.

Intermediário Usa Kafka básico, Feign/WebClient e contratos simples.

Avançado Implementa DLQ, retry, idempotência, Schema Registry, gRPC e tracing.

Sênior Decide entre eventos, REST, gRPC e GraphQL conforme acoplamento, escala e operação.

Testes e qualidade

Nível Domínio esperado

Iniciante Escreve testes unitários simples.

Intermediário Testa services, controllers e repositories.

Avançado Usa Testcontainers, WireMock, Pact, ArchUnit, mutation e carga.

Sênior Define estratégia por risco, qualidade de release e proteção arquitetural.

5. Aplicação prática em projetos reais

APIs

Aplicação:

- API de pedidos.
- API de clientes.
- API de produtos.
- API de pagamento.
- API de autenticação.

Boas práticas:

- OpenAPI.
- Versionamento.
- Erros padronizados.
- Idempotência.
- Paginação cursor.
- Rate limiting.
- Logs e métricas.
- Banco de dados

Aplicação:

- Pedidos.
- Faturas.
- Transações financeiras.
- Estoque.
- Auditoria.

Boas práticas:

- Índices adequados.
- Queries por caso de uso.
- Migrations Flyway.
- Lock otimista.
- Transações pequenas.
- Pool dimensionado.
- Observação de queries lentas.
- Microserviços

Aplicação:

- Orders Service.
- Payments Service.
- Inventory Service.
- Notification Service.
- Invoice Service.

Boas práticas:

- Dados próprios por serviço.
- Contratos explícitos.

- Observabilidade distribuída.
- Resiliência.
- Idempotência.
- Deploy independente.
- Evitar microserviço sem fronteira clara.
- Mensageria

Aplicação:

- Pedido criado.
- Pagamento aprovado.
- Nota emitida.
- Entrega iniciada.
- Cliente notificado.

Boas práticas:

- Eventos versionados.
- Schema Registry.
- DLQ.
- Retry topic.
- Reprocessamento.
- Consumer idempotente.
- Monitoramento de lag.
- Segurança

Aplicação:

- Login.
- APIs protegidas.
- Escopos.
- Roles.
- Integrações entre serviços.

Boas práticas:

- OAuth2/OIDC.
- JWT validado corretamente.
- Princípio do menor privilégio.
- Sanitização de entrada.
- Não expor stack trace.
- Rate limiting.
- Auditoria.
- Observabilidade

Aplicação:

- Logs estruturados.
- Métricas de latência.
- Tracing distribuído.
- Health checks.
- Alertas.

Boas práticas:

- Correlation ID.
- Métricas de negócio.
- Métricas técnicas.
- Dashboard por serviço.
- Alertas por sintoma, não apenas por causa.
- Testes

Aplicação:

- Domínio.
- API.
- Banco.
- Integração externa.
- Contrato.
- Performance.
- Arquitetura.

Boas práticas:

- Test pyramid.
- Testcontainers.
- WireMock.
- Pact.
- ArchUnit.
- k6/Gatling.
- Mutation testing.
- Deploy

Aplicação:

- Docker.
- Kubernetes.
- CI/CD.
- Blue/green.
- Canary.
- Rollback.

Boas práticas:

- Health checks.
- Readiness/liveness.
- Config por ambiente.
- Secrets seguros.
- Observabilidade no deploy.
- Rollback planejado.

6. Cronograma sugerido de execução

Abaixo está um cronograma de 40 semanas, pensado para estudo consistente e prática real. Pode ser comprimido ou expandido conforme sua carga horária.

Fase 1 - Semanas 1 a 6: Java moderno e JVM

Prioridade

Alta. Base obrigatória.

Estudar

- Java 17/21.
- Records.
- Sealed classes.
- Pattern matching.
- Streams.
- Generics.
- Concorrência.
- Virtual threads.
- JVM, GC, JFR e JMC.
- Prática
- Java Runtime Lab.
- Diagnóstico com heap/thread dump.
- Comparação entre threads tradicionais e virtual threads.
- Checkpoint

Explicar tecnicamente:

- Quando usar virtual threads.
- Como detectar memory leak.
- Como analisar thread dump.
- Como evitar uso ruim de Optional e Stream.

Fase 2 - Semanas 7 a 12: Spring Boot profissional e API Design

Prioridade

Alta. É a base para aplicação backend real.

Estudar

- Spring Boot 3.
- DI.
- Bean lifecycle.
- Configuration properties.
- Validation.
- Exception handling.
- Problem Details.
- OpenAPI.
- Rate limiting.

- Versionamento.
- Feign/WebClient.
- Resilience4j.
- Prática
- Order Management API.
- API com OpenAPI.
- Tratamento de erro padronizado.
- Integração externa com WireMock.
- Timeout, retry, circuit breaker.
- Checkpoint

Construir uma API que tenha:

- Contrato claro.
- Validação.
- Segurança básica.
- Observabilidade.
- Resiliência.
- Testes.

Fase 3 - Semanas 13 a 18: Persistência, banco e testes

Prioridade

Muito alta. Grande parte dos problemas reais está em dados, transações e queries.

Estudar

- JPA/Hibernate avançado.
- N+1.
- EntityGraph.
- Fetch join.
- Transações.
- Isolation levels.
- Flyway.
- HikariCP.
- Query tuning.
- Índices.
- Testcontainers.
- Prática
- Billing Persistence Lab.
- Teste com PostgreSQL real.
- Simulação de concorrência.
- Otimização de query lenta.
- Ajuste de pool.
- Checkpoint

Conseguir:

- Resolver N+1.

- Explicar isolation levels.
- Criar migrations.
- Testar persistência.
- Otimizar query com base em plano de execução.

Fase 4 - Semanas 19 a 25: Arquitetura e DDD

Prioridade

Alta para evolução avançada/sênior.

Estudar

- Clean Architecture.
- Hexagonal Architecture.
- DDD.
- Aggregates.
- Value Objects.
- Domain Events.
- Modular Monolith.
- Microservices.
- Outbox.
- CQRS.
- Saga.
- ADR.
- C4.
- Prática
- Modular Commerce Platform.
- Módulos separados.
- Domínio sem Spring.
- Ports and Adapters.
- ADRs e diagramas C4.
- Checkpoint

Conseguir:

- Separar domínio de infraestrutura.
- Justificar modular monolith vs microservices.
- Modelar bounded contexts.
- Criar ADRs.
- Projetar fluxo com eventos.

Fase 5 - Semanas 26 a 33: Kafka, gRPC, observabilidade e resiliência

Prioridade

Alta para sistemas distribuídos.

Estudar

- Kafka.
- Partitions.
- Consumer groups.
- Offset.
- Rebalance.
- DLQ.
- Retry topics.
- Schema Registry.
- Kafka Streams.
- gRPC.
- Protobuf.
- Tracing.
- Metrics.
- Logs.
- Backpressure.
- Prática
- Async Order Platform.
- Pipeline de pedidos.
- DLQ e reprocessamento.
- Kafka Streams.
- gRPC entre dois serviços.
- Monitoramento de lag.
- Checkpoint

Conseguir:

- Criar consumer idempotente.
- Projetar retry sem loop infinito.
- Monitorar lag.
- Evoluir schema.
- Comparar REST, gRPC e eventos.

Fase 6 - Semanas 34 a 40: System Design e liderança técnica

Prioridade

Essencial para senioridade.

Estudar

- Escalabilidade.
- Cache.
- Load balancing.
- Sharding.
- Replicação.
- CAP.
- PACELC.

- Multi-region.
- RPO/RTO.
- Rate limiting.
- Feed systems.
- Notification systems.
- ADR.
- RFC.
- Postmortem.
- Code review.
- Incident command.
- Prática
- Desenhar sistema de notificações.
- Desenhar feed.
- Desenhar encurtador de URL.
- Criar 10 ADRs.
- Criar 5 RFCs.
- Criar 5 postmortems.
- Criar checklist de code review.
- Checkpoint

Conseguir:

- Apresentar arquitetura com trade-offs.
- Explicar custo, risco e escalabilidade.
- Conduzir revisão técnica.
- Documentar decisões.
- Simular resposta a incidente.

7. Projetos recomendados para portfólio

1. Order Management API

Foco:

- Spring Boot.
- REST.
- OpenAPI.
- Validação.
- Exception handling.
- Segurança.
- Testes.

2. Billing Persistence Lab

Foco:

- JPA/Hibernate.
- Transações.

- Flyway.
- Query tuning.
- Locks.
- Testcontainers.

3. Modular Commerce Platform

Foco:

- Arquitetura hexagonal.
- DDD.
- Modular monolith.
- Domain events.
- ADR.
- C4 Model.

4. Async Order Platform

Foco:

- Kafka.
- Outbox.
- DLQ.
- Retry.
- Idempotência.
- Observabilidade.
- Reprocessamento.
- 5. gRPC Service Mesh Lab

Foco:

- Dois ou mais serviços Java.
- gRPC.
- Protobuf.
- Deadlines.
- Interceptors.
- Tracing.
- Comparação REST vs gRPC.

6. URL Shortener em escala

Foco:

- System Design.
- Cache.
- Rate limiting.
- Banco.
- Consistent hashing conceitual.
- Observabilidade.
- Teste de carga.

7. Notification System

Foco:

- Fan-out.
- Kafka.
- WebSocket/SSE.
- Push.
- Backpressure.
- Retry.
- Multi-canal.

8. Checklist de domínio técnico

Java

- [] Sei usar Java 17/21 de forma idiomática.
- [] Sei aplicar records, sealed classes e pattern matching.
- [] Sei usar generics avançado.
- [] Sei evitar mau uso de Optional e Stream.
- [] Sei explicar virtual threads.
- [] Sei analisar heap dump e thread dump.
- [] Sei usar JFR/JMC para diagnóstico.
- Spring Boot
- [] Sei criar API com validação e erros padronizados.
- [] Sei configurar profiles e properties.
- [] Sei usar Actuator e métricas.
- [] Sei criar integração externa resiliente.
- [] Sei configurar timeout, retry e circuit breaker.
- [] Sei aplicar segurança em APIs.
- [] Sei diferenciar erro de negócio, técnico e transitório.
- APIs
- [] Sei desenhar contratos REST.
- [] Sei documentar com OpenAPI.
- [] Sei aplicar idempotência.
- [] Sei escolher paginação offset ou cursor.
- [] Sei usar status HTTP corretamente.
- [] Sei planejar versionamento e depreciação.
- [] Sei justificar REST vs gRPC vs GraphQL vs eventos.
- Persistência
- [] Sei resolver N+1.
- [] Sei usar fetch join e EntityGraph.
- [] Sei explicar persistence context e dirty checking.
- [] Sei configurar transações.
- [] Sei explicar isolation levels.

- ☐ Sei ajustar HikariCP.
- ☐ Sei otimizar query com índice.
- ☐ Sei aplicar Flyway.
- ☐ Sei usar outbox.
- Arquitetura
- ☐ Sei aplicar arquitetura hexagonal.
- ☐ Sei separar domínio, aplicação, infraestrutura e interface.
- ☐ Sei modelar aggregates e value objects.
- ☐ Sei definir bounded contexts.
- ☐ Sei decidir modular monolith vs microservices.
- ☐ Sei escrever ADR.
- ☐ Sei desenhar C4 Model.
- Kafka e mensageria
- ☐ Sei criar producer e consumer.
- ☐ Sei explicar partitions e consumer groups.
- ☐ Sei tratar offset e rebalance.
- ☐ Sei configurar DLQ.
- ☐ Sei implementar retry topic.
- ☐ Sei criar consumer idempotente.
- ☐ Sei usar Schema Registry.
- ☐ Sei monitorar lag.
- ☐ Sei usar Kafka Streams para agregações.
- Testes
- ☐ Sei escrever testes unitários bons.
- ☐ Sei usar Mockito sem acoplar teste à implementação.
- ☐ Sei testar API.
- ☐ Sei usar Testcontainers.
- ☐ Sei usar WireMock.
- ☐ Sei aplicar contract testing.
- ☐ Sei criar teste de carga.
- ☐ Sei usar ArchUnit.
- ☐ Sei definir estratégia de testes por risco.
- System Design
- ☐ Sei desenhar sistema escalável.
- ☐ Sei explicar cache e invalidação.
- ☐ Sei aplicar rate limiting.
- ☐ Sei discutir CAP/PACELC.
- ☐ Sei explicar sharding e replicação.
- ☐ Sei planejar RPO/RTO.
- ☐ Sei considerar custo, operação e falhas.
- ☐ Sei desenhar sistemas de feed, notificações e pedidos assíncronos.
- Liderança técnica
- ☐ Sei conduzir code review efetivo.
- ☐ Sei escrever RFC.

- [] Sei escrever ADR.
- [] Sei criar postmortem.
- [] Sei comunicar risco técnico.
- [] Sei negociar escopo.
- [] Sei mentorar outros devs.
- [] Sei influenciar tecnicamente sem depender de cargo.

9. Indicadores de prontidão para nível avançado/sênior

Você começa a demonstrar maturidade avançada/sênior quando consegue:

- Projetar antes de codar
- Entende requisitos, restrições, trade-offs e riscos.
- Justificar decisões
- Não escolhe tecnologia por moda, mas por contexto.
- Pensar em produção
- Considera logs, métricas, tracing, falhas, rollback e custo operacional.
- Evitar acoplamento desnecessário
- Protege domínio de framework, banco e integrações externas.
- Criar sistemas evolutivos
- APIs versionáveis, eventos compatíveis, banco migrável e arquitetura modular.
- Diagnosticar problemas reais
- CPU, memória, GC, banco, Kafka lag, timeout, pool e latência.
- Definir qualidade por risco
- Sabe onde precisa de teste unitário, integração, contrato, carga ou arquitetura.
- Lidar com sistemas distribuídos
- Entende consistência eventual, idempotência, retry, DLQ, saga e outbox.
- Comunicar tecnicamente
- Escreve ADRs, RFCs, postmortems e explica trade-offs para diferentes públicos.
- Elevar o time
- Melhora padrões, revisa código, orienta devs e reduz débito técnico.